Data Encryption Standard (DES) Algorithm

Aim:

To implement the Data Encryption Standard (DES) algorithm for symmetric key encryption.
To perform encryption and decryption of data using a shared secret key.

Description:

Data Encryption Standard (DES) is a symmetric-key block cipher developed by IBM and standardized by NIST. It operates on 64-bit blocks of plaintext using an effective 56-bit key, with 8 additional parity bits.
DES uses a 16-round Feistel structure, where each round performs substitution and permutation operations. The encryption process involves an initial permutation, 16 rounds of processing, and a final permutation. Decryption uses the same algorithm as encryption but applies the round keys in reverse order. Although historically important, DES is now considered insecure because its 56-bit key can be brute-forced with modern computing resources.

Algorithm

1. Input the 64-bit plaintext and the 64-bit DES key, where 8 bits of the key are used for parity.
2. Apply the Initial Permutation (IP) to the plaintext and divide the result into two 32-bit halves, L0 and R0.
3. Generate 16 round keys of 48 bits from the 56-bit effective key using permutation, splitting, and circular left shifts.
4. Perform 16 Feistel rounds using:

$$L_i = R_{i-1}$$
$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Where F includes expansion, XOR with the round key, S-box substitution, and permutation.

5. After the 16th round, interchange the two halves and apply the Final Permutation (IP^{-1}) to obtain the ciphertext.
6. For decryption, repeat the same process using the 16 round keys in reverse order to recover the original plaintext.

Flowcharts

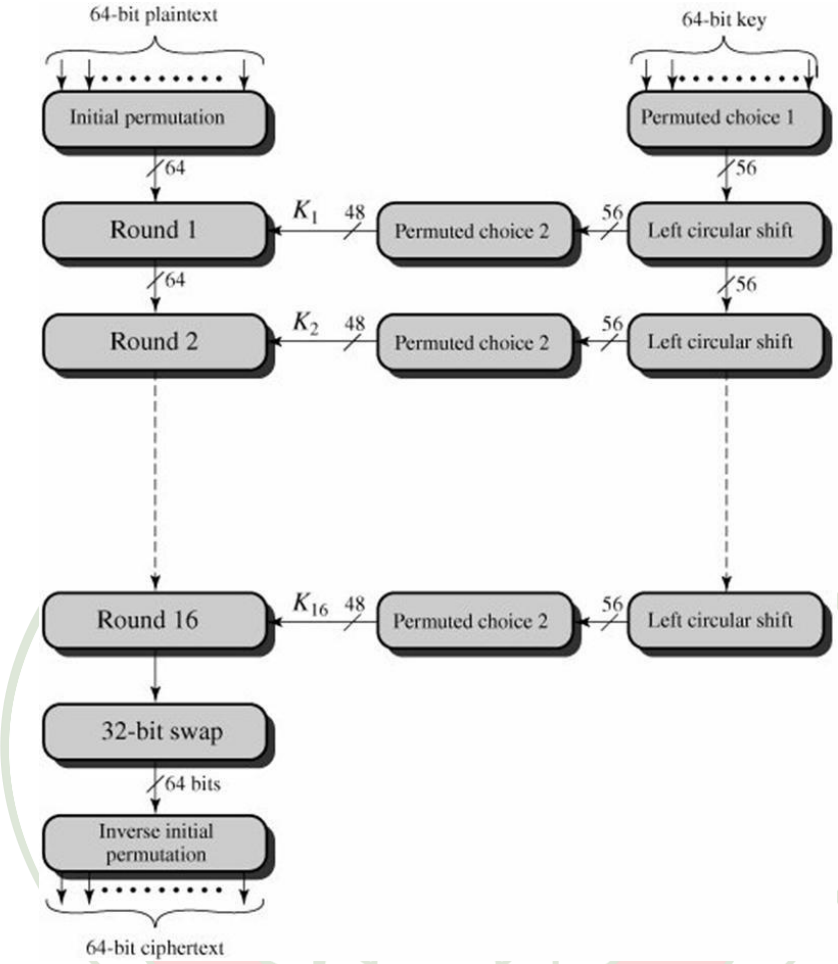


Fig. DES

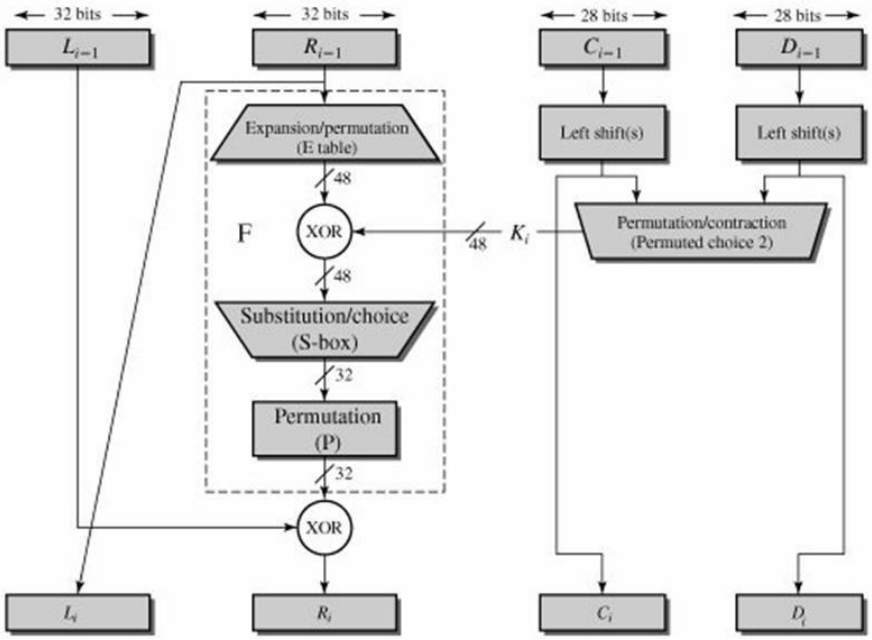


Fig. Single Round of DES

Implementation

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b"12345678"
cipher = DES.new(key, DES.MODE_ECB)
plaintext = "Hello DES"

encrypted = cipher.encrypt(pad(plaintext.encode(), DES.block_size))
decipher = DES.new(key, DES.MODE_ECB)
decrypted = unpad(decipher.decrypt(encrypted), DES.block_size)

print("Encrypted:", encrypted.hex())
print("Decrypted:", decrypted.decode())
```

Output

```
Encrypted: e9167aa16c791f5b0072b98577787088
Decrypted: Hello DES
```

Result Analysis

- The DES algorithm successfully performs encryption and decryption using a common symmetric secret key.
- DES processes data in fixed 64-bit blocks and uses a 16-round Feistel structure.
- The combination of substitution through S-boxes and permutation provides confusion and diffusion in the ciphertext.
- The same DES structure can be used for both encryption and decryption by reversing the order of round keys.
- The experiment demonstrates the importance of secure key generation and management in symmetric-key cryptography.
- DES is computationally efficient compared with many asymmetric algorithms but its 56-bit effective key is insufficient against modern brute-force attacks.
- DES has therefore been replaced in practical applications by stronger standards such as AES.

Conclusion

The DES symmetric-key encryption algorithm was implemented and its encryption and decryption process was studied. The experiment demonstrated the working of a 16-round Feistel cipher and the role of symmetric keys in secure communication.